

Medical Guideline Assistant

A retrieval-augmented assistant that answers only from official MOHFW Standard Treatment Guidelines — and refuses when it cannot.

20 DOCUMENTS	3,414 CHUNKS	3 SAFETY GATES	100 TESTS	~5.5k LINES OF PYTHON
------------------------	------------------------	--------------------------	---------------------	---------------------------------

This system answers clinical questions strictly from a curated corpus of Indian government treatment guidelines, with a citation behind every claim. Its defining property is not that it answers well — it is that it **declines predictably**. Three independent gates sit around the model, and any one of them can stop an answer from reaching the user.

1. What the system is

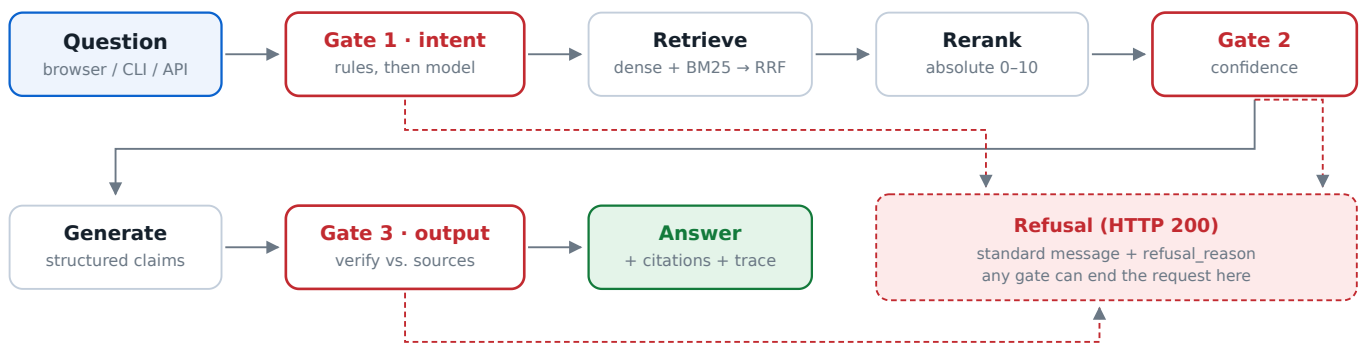
A user asks a question in a browser. The system decides whether the question is even in scope, retrieves passages from a vector store and a lexical index, scores how well each passage actually answers the question, refuses if none of them do, generates an answer constrained to those passages, and then verifies the generated text against them before showing it. The same pipeline backs a CLI and a JSON API.

The corpus is deliberately narrow: 20 documents, hash-verified against a manifest, covering tuberculosis, hypertension, malaria, antimicrobial use, critical care, paediatrics, obstetrics and related areas. Anything outside that is refused rather than answered from the model's own knowledge.

Three questions the design keeps answering

Is this question in scope?	Gate 1 decides before any retrieval runs, so an out-of-scope query never costs an embedding call and a personal one never has guideline text pulled up beside it.
Do we actually have the answer?	Gate 2 reads an <i>absolute</i> relevance score, not a ranking. The best of five irrelevant passages still ranks first; only an absolute score can express “nothing here answers this”.
Is the answer supported?	Gate 3 re-reads the draft against the passages it cites, checking citations, numbers, claim support and framing before release.

2. The pipeline



Cache (Upstash Redis)

L0 query embeddings · L1 whole responses, keyed by a pipeline fingerprint · read-through, never fails closed
A fail-closed refusal is never cached: an outage must not be pinned into Redis for the whole TTL.

Every exit produces the same `Response` object. A refusal is a 200 with `answered: false`, never an HTTP error — refusing is a correct outcome, and a 4xx would invite callers to retry around it.

3. Repository structure

Rag-project/	
├─ app.py	Vercel ASGI entrypoint – puts <code>src/</code> on the path, re-exports the app
├─ vercel.json	function config: bundle contents, memory, max duration
├─ .vercelignore	keeps <code>.env</code> , <code>.venv</code> and the 89 MB embed cache out of uploads
├─ pyproject.toml	runtime deps; <code>`ingest`</code> extra holds the 64 MB PDF parser
├─ src/rag_project/	
│ ├─ config.py	every tunable; thresholds load from <code>data/calibration.json</code>
│ ├─ models.py	shared schemas – <code>Chunk</code> , <code>Retrieved</code> , <code>Claim</code> , <code>Response</code>
│ ├─ llm.py	OpenAI wrapper; failures raise, callers decide
│ ├─ cache.py	two-tier read-through cache over Upstash
│ ├─ security.py	password gate (fails closed) + rate limit (fails open)
│ ├─ assistant.py	the pipeline, end to end
│ ├─ api.py	FastAPI service and the web UI it serves
│ └─ cli.py	<code>`rag`</code> – corpus, ingest, index, ask, eval, store
├─ corpus/manifest.py	sha256 document allow-list
├─ ingest/	PDF → sections → chunks
│ ├─ parse.py	layout-aware PDF parsing (<code>pymupdf</code> , imported lazily)
│ ├─ text.py	plain-text source handling
│ ├─ chunk.py	section-aware chunking, 320 tokens / 48 overlap
│ └─ pipeline.py	orchestration + <code>load_chunks()</code> used at query time
├─ indexing/	
│ ├─ embed.py	OpenAI embeddings; npz for ingest, Redis for queries
│ ├─ bm25.py	lexical index + reciprocal rank fusion
│ ├─ store.py	LanceDB over plain HTTP (<code>httpx</code> , no heavy client)
│ └─ build.py	embeds chunks and pushes the table
├─ retrieval/	
│ ├─ rewrite.py	curated abbreviation glossary; expansions are additive
│ ├─ search.py	hybrid dense + lexical retrieval, fused on rank
│ └─ rerank.py	absolute 0–10 relevance scoring
├─ generation/	
│ ├─ prompt.py	the grounded-answer system prompt
│ └─ generate.py	structured generation into <code>GroundedAnswer</code>

├─ guardrails/	
│ └─ policy.py	scope statement, disclaimer, refusal messages
│ └─ input_gate.py	Gate 1 – intent classification
│ └─ confidence.py	Gate 2 – absolute-score threshold
│ └─ output_gate.py	Gate 3 – citations, numbers, support, framing
├─ evaluation/	eval set, runner, threshold calibration
├─ web/static/	
│ └─ index.html	the single-page UI (no build step, no CDN)
│ └─ login.html	password gate page
├─ data/	
│ └─ corpus_manifest.yaml	the allow-list: a PDF not listed here is never ingested
│ └─ chunks/*.jsonl	committed – BM25 reads these at boot
│ └─ calibration.json	committed – the measured gate-2 threshold
│ └─ index_manifest.json	committed – what produced the table being queried
│ └─ embed_cache/	89 MB, gitignored, ingest only
│ └─ raw/	source PDFs, gitignored
└─ tests/	100 tests across 7 files

Why some data files are committed

`data/chunks/`, `calibration.json` and `index_manifest.json` are tracked deliberately. The lexical half of hybrid search reads the chunks at boot, so a deployment without them raises `RetrievalError` and refuses every question. The calibration file carries the gate-2 threshold that was actually measured (5.5) rather than the provisional default in code (5.0). Together they are 4.6 MB. The 89 MB embed cache is *not* committed — it is ingest infrastructure and never touched on the query path.

4. Module reference

MODULE	LINES	RESPONSIBILITY
<code>config.py</code>	159	Single source of tunables. Also computes <code>pipeline_fingerprint</code> , which namespaces the response cache so a reindex or recalibration makes old entries unreachable rather than stale.
<code>models.py</code>	113	Contracts shared by every stage. <code>Claim.chunk_ids</code> has <code>min_length=1</code> , so the schema itself forbids an uncited claim.
<code>llm.py</code>	103	One place for model choice, retries, and failure policy. An LLM error raises; it never degrades into “answer anyway”. Also records <i>degradations</i> so a fail-closed refusal is distinguishable from a considered one.
<code>assistant.py</code>	187	The pipeline. <code>ask()</code> wraps caching around <code>_run()</code> , which walks the six stages and builds the trace.
<code>cache.py</code>	221	Bounded in-process LRU in front of Upstash, short timeouts, circuit breaker after 3 consecutive failures.
<code>security.py</code>	153	HMAC session cookies keyed off the password itself, plus per-IP rate limiting on separate counters for asking and logging in.
<code>api.py</code>	214	FastAPI app: serves the UI at <code>/</code> , the API under <code>/api/</code> , login/logout, and an unauthenticated <code>/health</code> .
<code>ingest/parse.py</code>	316	Layout-aware PDF parsing — heading detection by font size with a rescue rule for headings typeset at body size, plus running-header removal.

guardrails/output_gate.py	329	The largest guardrail: citation validity, unsupported numbers, claim-level support judging, and framing checks, with partial repair.
retrieval/rewrite.py	94	Hand-curated abbreviation glossary. Expansions are additive and ambiguous abbreviations are deliberately left alone.

5. Request flow, end to end

5.1 A refused question

```
POST /api/ask {"query": "Should I take rifampicin for my cough?"}

api.require_auth      session cookie valid (or no password configured)
api.enforce_rate_limit INCR rl:ask:<window>:<hash(ip)> → within quota
Assistant.ask         cache miss → _run()
  input_gate.classify  regex "\bshould i\b" matches
                      → Intent.PERSONALIZED_ADVICE
  x short-circuit      no retrieval, no embedding, no model call

Response(answered=False, refusal_reason="personalized_advice",
          trace={"stages": ["intent"], "short_circuited": "no retrieval performed"})
```

Rules run first and may only ever *escalate* to a refusal — they are never allowed to stamp a query in-scope. If they could, any phrasing the regexes failed to anticipate would bypass the model check entirely, which is how a blocklist quietly becomes an allowlist.

5.2 An answered question

```
POST /api/ask {"query": "What does the guideline say about TB treatment duration?"}

Assistant.ask          cache key = sha256(casefolded query, screen) under fingerprint
├ HIT → return stored Response, trace.cache = "hit"           ~107 ms
└ MISS → _run()                                              ~10 s

Gate 1 input_gate      no rule match → model classifies → IN_SCOPE
Retrieve search.py     rewrite: "TB" → "TB tuberculosis" (additive)
                      dense: embed query → LanceDB top-20
                      lexical: BM25 over 3,414 local chunks → top-20
                      fuse: reciprocal rank fusion → 34 candidates

Rerank rerank.py       one batched call scores each passage 0–10, absolutely
Gate 2 confidence      top score 9.0 ≥ threshold 5.5 → keep 6, label C1...C6
Generate generate.py   structured GroundedAnswer: prose + claims + chunk_ids
Gate 3 output_gate     citations valid? numbers present in cited text?
                      each claim supported? framing third-person?
                      → strips 1 unsupported claim, passes the rest

Response(answered=True, citations=[C1...C6], trace={...})
→ cached for 24 h, unless any stage was degraded
```

5.3 What the browser does with it

The UI requests `trace: true` and renders a **pipeline panel** under every answer showing what each stage decided. That panel is the reason the UI exists: a chat box shows you an answer, but it cannot show you that a question was refused *before retrieval ran*, or that the reranker scored every passage zero. Hovering a `[C1]` marker highlights the source it refers to.

6. Offline flow — corpus to index

rag corpus scan	inspect data/raw/, write candidate manifest entries (url, specialty and version left blank rather than guessed)
rag ingest	verify sha256 against corpus_manifest.yaml, then parse.py → sections → chunk.py → data/chunks/*.jsonl
rag index	embed.py (npz cache) → store.py → LanceDB table
rag eval calibrate	measure the gate-2 threshold from labelled cases
rag eval run	full pipeline metrics: safety compliance, false-refusal rate

CORPUS GOVERNANCE

A PDF sitting in `data/raw/` is not part of the corpus. A PDF **listed in the manifest with a matching sha256** is. Dropping an uncurated file into the directory halts ingestion with a named error rather than quietly widening what the assistant will speak about.

Threshold calibration

Gate 2's threshold is measured, not chosen. `rag eval calibrate` stops after reranking — it only needs the top-1 score per question, so it skips generation and costs a fraction as much. It then sweeps candidate thresholds and takes the *midpoint of the widest maximum-accuracy band* rather than an edge, because rerank scores move slightly between runs and a threshold sitting exactly on an observed score flips that case on the next query. The measured value is 5.5, against answerable cases scoring 10.0 and unanswerable ones 0.0.

7. Caching

LAYER	KEY	TTL	PURPOSE
L0	emb:sha256(model,dim,text)	30 d	Query embeddings. Also removes an 89 MB npz rewrite from the hot path — the disk cache is now lazy and ingest-only.
L1	resp:<fingerprint>:sha256(query)	24 h	Whole responses. A hit skips all five model calls: ~10 s → ~107 ms.
rate	rl:<scope>:<window>:hash(ip)	1 h	Per-IP counters, one <code>INCR</code> + <code>EXPIRE</code> pipeline per check.

Two rules, inverted from the rest of the system

- **The cache never fails closed.** Everything in `llm.py` refuses on error because every caller there is a safety gate. Nothing in the cache is. A miss, a timeout or an unreachable Upstash all degrade to “compute it normally”.
- **The cache never adds latency.** Upstash is a remote TLS hop of tens of milliseconds. Timeouts are short and retries are disabled — redis-py retries three times with backoff by default, which silently turned a 0.4 s budget into a measured 0.73 s.

OUTAGES ARE NEVER CACHED

When a stage falls back because a model was unreachable, it records itself via `llm.note_degraded()`. `Assistant.ask` refuses to store any response carrying a degradation — otherwise a minute of API trouble becomes a full day of a good question being refused from cache.

8. Access control

CONTROL	FAILS	BEHAVIOUR
Password gate APP_PASSWORD	closed	Unset by default, so local work and tests need nothing. When set, <code>/</code> serves a login page and every <code>/api</code> route returns 401. Session is an HMAC-signed cookie — <code>httponly</code> , <code>samesite=lax</code> , and <code>secure</code> only over real HTTPS. The signing key derives from the password, so rotating it logs everyone out.
Rate limit RATE_LIMIT_PER_WINDOW	open	Per-IP fixed window, default 30/hour, IP hashed not stored. Falls back to a per-process counter when Redis is unreachable. Asking and logging in use <i>separate</i> counters, so failed logins never consume the question quota.
Health check	—	<code>/health</code> stays open: uptime monitoring must not need a password, and it reveals nothing but liveness.

The API key itself is never exposed — it appears in no response model, trace field or log line, and all model calls are server-side. Neither control replaces a hard monthly spend limit on a project-scoped OpenAI key, which is the only backstop no application bug can undo.

9. Deployment

CONCERN	RESOLUTION
Runtime bundle	124 MB installed against Vercel's 250 MB limit. The PDF parser (64 MB) sits in the <code>ingest</code> extra and its import is lazy, so a deploy that only answers questions never loads it.
Vector store	Self-hosted LanceDB reached over plain <code>httpx</code> , so the heavy client and its Arrow dependency never enter the bundle.
Cold start	~2.2 s to import and build the BM25 index over 3,414 chunks; 204 MB peak RSS against 1024 MB requested.
Entrypoint	Top-level <code>app.py</code> , which Vercel auto-detects; <code>src/</code> is added to <code>sys.path</code> so the import works whether or not the package was installed.
Required config	<code>OPENAI_API_KEY</code> , <code>LANCEDB_URI</code> , <code>LANCEDB_API_KEY</code> , <code>REDIS_URL</code> . Everything else matches a code default.

10. Testing

SUITE	TESTS	
test_security.py	22	Tests are isolated from live infrastructure: an autouse fixture clears <code>REDIS_URL</code>, because <code>.env</code> carries a real one and the suite was otherwise reading and writing production keys — rate-limit counters survived between runs, which made the limiter tests pass alone and fail together. <code>test_web_ui.py</code> guards the Python/JavaScript seam. The pipeline panel is rendered in the browser now, so those tests compare the JS stage table against the keys <code>assistant.py</code> actually writes. Renaming a trace key
test_cache.py	19	
test_guardrails.py	16	
test_ingest.py	13	
test_api.py	10	
test_web_ui.py	9	

test_indexing.py	5
Total	100

would not raise — the panel would silently show “not reached”.

11. Decisions worth knowing

- **Rank fusion, not score fusion.** Dense distances and BM25 scores are not on a comparable scale, so hybrid retrieval fuses on rank.
- **Absolute rerank scores.** Gate 2 refuses on the top-1 value, so the prompt asks how well a passage answers the question on a fixed scale — never which passage is best.
- **Additive query expansion.** “TB” becomes “TB tuberculosis”, never “tuberculosis”: replacing the token would destroy exactly what BM25 matches on. Ambiguous abbreviations (`MI`, `DM`, `BP`) are left alone, because guessing wrong in a medical corpus is not recoverable.
- **Refusals are 200s.** Encoding them as 4xx would push callers toward treating a correct outcome as a fault to be retried around.
- **The evaluation view has no “run” button.** A run costs real API calls and takes minutes — too easy to trigger by accident from a browser.
- **Blank provenance over invented provenance.** `corpus scan` leaves url, specialty and version empty rather than guessing; those fields end up in citations, and a blank is visible where a fabrication is not.

Generated from the repository at commit time · 100 tests passing · figures and measurements in this document were taken from the running system.